웹서비스설계 보고서 작성 양식

JWT 게시판 구현 보고서

01 서비스 소개와 구현 범위

학번 [202212950] 이름: [백종학] 제출일: [2026.10.28]

서비스 이름 [자취생 정보공유 게시판]

기술 조합과 버전 [기술조합: FastAPI(0.143.0)/React(19.3.0) OS: Windows 11 DB: SQLite(3.35.5)]

GitHub 저장소 URL [https://github.com/jongka/web_service_design_JWT-Assignment_202212950]

소스 코드 기준 [main/62e2e84b74a45bd407bcb1e6c5d1cbb5637334d8.]

서비스의 대상 사용자와 사용 목적

'자취생 전용 정보공유방' 게시판은 자취생들을 사용자로 각자의 생활 팁이나 일상을 공유하기 위한 목적으로 사용한다.

기술 조합은 Python 을 기반으로 API 를 구현하고자 FastApi 를 사용했고 코드의 재사용성을 고려하여 컴포넌트 단위로 개발할 수 있는 React 를 사용했다. 게시글은 SQLite 파일에 저장된다.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	완료	app.py, store.py, main.jsx	
로그인과 JWT 발급	완료	users.py, auth.py, app.py	
인증과 내 정보 조회	완료	auth.py, app.py, api.js	
본인 글 작성·수정·삭제	완료	app.py, auth.py, store.py	
만료·로그아웃·오류 표시	완료	auth.py, api.js, main.jsx, style.css	
DB 데이터 유지	완료	store.py	

구조와 변경 내역

브라우저 → API 서버 → SQLite 의 연결 구조 및 로그인 요청과 게시글 수정 요청의 흐름

브라우저: React main.jsx
-> api.js: 로그인 상태 및 fetch 요청
-> FastAPI app.py(8000)
-> users.py: 계정조회,scrypt 비밀번호 확인
-> auth.py: JWT 발급, 검증, 작성자 비교
-> store.py: SQLite 게시글 CRUD

사용자가 브라우저 로그인 폼에 이메일과 비밀번호를 입력하면 api.js 의 login()이 호출되어 API 서버에 email, password 를 JSON 으로 전송한다. 이후 app.py 의 fields()가 입력 형식을 검사하고, users.py 의 verify_credentials()가 scrypt 로 계산한 비밀번호 해시를 compare_digest()로 비교한다. 계정 정보가 일치하면 issue_token()으로 JWT 를 발급한다. 프론트엔드는 응답받은 토큰과 공개 사용자 정보를 api.js 의 메모리 변수 session 에 저장한다.

게시글 등록·수정·삭제 요청에는 Authorization: Bearer <JWT> 헤더를 첨부한다. 신규 등록에서는 서버가 토큰을 검증하고 입력을 검사한 뒤, 인증 사용자 ID 를 작성자로 하여 SQLite 에 게시글을 저장한다.
수정·삭제에서는 토큰 검증 후 게시글의 존재를 확인하고, 저장된 authorId 와 인증 사용자 ID 를 비교한다. 작성자가 일치하면 수정을 위한 입력 검사를 거쳐 데이터를 변경하거나, 해당 게시글을 삭제한다.

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
backend-fastapi/auth.py: _settings	설정 처리	환경변수, 키·TTL 형식 검사 및 hex→bytes	설정 처리 미구현/AI 초안 생성 후 수정하여 반영
auth.py: issue_token	501 임시 오류	토큰 발급과 필수 클레임 구성	토큰 발급 구현/AI 초안 생성 후 수정하여 반영
auth.py: current_user_id	고정 ID 1	Bearer·서명·클레임·만료·실제 계정 검증	계정 검증 구현/AI 초안 생성 후 수정하여 반영
auth.py: assert_owner	권한 검사 없음	사용자와 작성자 비교하여 타인은 403	권한 검사 구현/AI 초안 생성 후 수정하여 반영
backend-fastapi/app.py	라우트·입력·응답 처리 제공	API 문서 제목을 서비스명으로 변경	제공 코드 유지하고 제목만 수정
frontend-react/src/api.js	인증 TODO, canEdit 항상 true	메모리 세션·로그인·Bearer·상태별 오류·작성자 비교	AI 초안 생성 후 수정하여 반영
rontend-react/src/main.jsx	게시판 기본 화면 제공	로그인·오류·내 정보·수정 상태·버튼 및 주제 반영	제공 화면에 주제에 맞춰 AI 수정 반영
frontend-react/src/style.css	기본 스타일	오류 메시지와 비활성 버튼 표현	제공 스타일에 주제에 맞춰 AI 수정 반영
frontend-react/index.html	기본 문서 제목	서비스명·설명 변경	주제에 맞춰 AI 수정 반영

로컬 실행 기록

[운영체제와 버전 / 새 폴더 설치 / 환경설정 / 프론트 빌드 / 백엔드 실행 / SQLite 파일 경로]

Windows11 의 PowerShell 환경에서 로컬 실행을 검증하였다.

소스를 바탕화면의 새 폴더에 복사하여 재현 검사를 진행하였다. DB 과 Python 캐시는 제외하였다.

새로운 Python 가상환경을 만들고 requirements.txt 를 설치하였다.

```
python -m venv .venv

.\.venv\Scripts\python.exe -m pip install -r requirements.txt

.\.venv\Scripts\python.exe -m pip check
```

```
# frontend-react 폴더

npm.cmd ci

npm.cmd run build

npm.cmd run preview
```

```
# backend-fastapi 폴더

.\.venv\Scripts\python.exe -m uvicorn app:app --host 127.0.0.1 --port 8000
```

SQLite 파일은 새 복사본의 backend-fastapi/data/board.sqlite 에 자동 생성되었다.

[4173 번 접속 및 API 연결 확인 / 서버 재시작 후 데이터 유지 / 증거 위치]

4173 번 preview 서버를 통해 api 요청을 보내 전달되는지 검사하였고 서버 재시작 후 데이터 유지 또한 검사 완료하였다. (증거: docs/evidence/T15-server_restart 및 T15-server_restart_log)

핵심 코드 설명

3 1 로그인과 JWT 발급

파일 / 함수: backend-fastapi/app.py 의 login(),

users.py 의 verify_credentials(),

auth.py 의 _settings()·issue_token()

코드 발췌

```
@app.post("/api/auth/login")
async def login(request: Request):
  data = await fields(request, dict(email=254, password=128))
  user = verify_credentials(**data)
  if not user:
    raise ApiError(401, "INVALID_CREDENTIALS", "이메일 또는 비밀번호를 확인하세요.")
  return dict(
    accessToken=issue_token(user),
    expiresIn=int(os.getenv("TOKEN_TTL", "90")),
    user=user,
  )
```

동작과 응답 코드 설명

입력으로 email 과 password 문자열이 들어온다. fields()는 허용되지 않은 값이나 형식을 400 BAD_INPUT 으로 거부한다. 비밀번호 scrypt 해시를 계산하고 저장된 해시와 비교하여 계정을 확인한다. verify_credentials()가 계정 또는 비밀번호 불일치를 반환하면 401, INVALID_CREDENTIALS 를 발생시키고 토큰을 발급하지 않는다.

계정이 존재하고 password 가 일치하면 토큰이 발급되며 기본 90 초의 만료시간을 가지는데 설정에 따라 변경 가능하다. 성공 시 HTTP 200, accessToken, expiresIn, user 를 반환한다.

3 2 브라우저 토큰 저장과 전송

파일 / 함수: frontend-react/src/api.js 의 login(), authorizationHeaders(), clearSession(), request()

코드 발췌

```
export async function login(email, password) {
 const data = await request("/auth/login", {
  method: "POST",
  body: JSON.stringify({ email, password }),
 });
 session = { accessToken: data.accessToken, user: data.user };
 return session.user;
}


function authorizationHeaders() {
 return session.accessToken
  ? { Authorization: `Bearer ${session.accessToken}` }
  : {};
}
```

동작과 응답 코드 설명

로그인 성공 응답(HTTP 200)을 받은 후 메모리 변수 session 에 accessToken 과 user 를 저장한다. 별도로 session 을 저장하지 않기 때문에 새로고침하면 세션은 초기화된다. 토큰이 있을 때만 Authorization 헤더를 만든다. 공통 request()가 헤더를 추가한다.

로그인 실패는 HTTP 401 과 INVALID_CREDENTIALS 로 응답하고 보호 API 에서 401 을 받으면 TOKEN_REQUIRED, INVALID_TOKEN, TOKEN_EXPIRED 중 맞는 걸 표시하고 세션을 제거, 403 과 FORBIDDEN 은 권한 부족이므로 로그인을 유지한다.

로그아웃은 clearSession()과 React 사용자, 프로필, 수정 상태를 초기화로 처리한다. 별도의 서버 요청 없으므로 HTTP, JSON 은 없다. 이미 복사된 JWT 를 즉시 무효화 하지는 않는다.

서버 인증과 작성자 권한

3 3 서버 JWT 검증

파일 / 함수: backend-fastapi/auth.py 의 current_user_id()

코드 발췌

```
claims = jwt.decode(
    match.group(1), key, algorithms=["HS256"],
    issuer=issuer, audience=audience,
    options={"require": ["sub", "exp", "iss", "aud"],
          "verify_exp": False, "strict_aud": True},
)
subject = claims["sub"]
expiration = claims["exp"]
...
if expiration <= time.time():
    raise ApiError(401, "TOKEN_EXPIRED", "로그인 시간이 만료되었습니다. 다시 로그인하세요.")
return user_id
```

동작과 응답 코드 설명

Authorization 이 Bearer 형식인지 확인하고 없거나 형식이 틀리다면 HTTP 401, TOKEN_REQUIRED 응답한다. 허용 알고리즘을 HS256 으로 고정하고 키, 발급자, 대상을 전달하여 서명과 클레임을 검증한다. require sub, exp, iss, aud 가 반드시 있도록 한다.

필수 클레임 형식이 잘못된 오류 또는 존재하지 않는 사용자는 401 INVALID_TOKEN 으로 거부한다.

현재 코드에서 verify_exp=False 로 자동 만료 검사를 끄고 아래쪽 코드로 직접 만료를 비교 판단한다. 검증이 끝난 사용자 ID 만 /me 와 글 변경 라우트로 전달한다. 만약 만료 되었다면 HTTP 401, TOKEN_EXPIRED 를 반환한다. 성공 시에는 HTTP 200 과 공개 user 정보를 반환한다.

3 4 작성자 권한 검사

파일 / 함수: backend-fastapi/auth.py 의 assert_owner(),

app.py 의 create()·update()·delete()

코드 발췌

```
def assert_owner(user_id, author_id):
  if user_id != author_id:
    raise ApiError(403, "FORBIDDEN", "본인이 작성한 글만 수정하거나 삭제할 수 있습니다.")


@app.put("/api/posts/{id}")
async def update(id: str, request: Request):
  user_id = current_user_id(request)
  pid = post_id(id)
  post = existing(pid)
  assert_owner(user_id, post["authorId"])
  return store.update_post(pid, await fields(request, dict(title=100, content=5000)))
```

동작과 응답 코드 설명

토큰 검증으로 얻은 현재 사용자 ID 와 DB 에서 조회한 글의 authorId 를 비교한다. 인증에 실패하면 401 이고 인증은 통과했지만 작성자가 다르면 403 을 띄운다. 권한 확인이 입력 검사와 DB 변경보다 먼저이므로 Bob 이 Alice 글에 제목을 보내도 403 이 반환된다. 토큰이 없으면 글 존재 확인 전 401 이 먼저 반환되고 정상 토큰으로 없는 글에 접근하면 404 를 반환한다.

current_user_id()결과를 store.create_post()에 전달한다. 요청은 title, content 만 허용하고 authorId, userId, id 같은 추가 필드를 거부하여 작성자 위조를 막는다. canEdit()로 버튼 감추기 처리와 별개로 서버 검사도 항상 수행한다.

테스트 결과

T01 부터 T17 까지 수행하고, 실제 응답과 증거의 위치를 기록하세요.

실행 환경: [OS / 런타임 버전 / 브라우저 / 실행 명령]

로컬 배포 확인: [새 폴더에서 설치한 절차 / 프론트 빌드 결과 / 백엔드 실행 명령 / 4173 번 접속 및 API 연결 증거]

테스트 일시 및 코드 버전: [작성]

토큰 TTL: [일반 테스트:500/만료 테스트: 3]

05_테스트시나리오.md 의 T01~T17 결과를 모두 기록하세요. PASS, FAIL, 미실행을 구분합니다. 예상 결과를 실제 결과로 복사하지 말고 직접 관찰한 HTTP 상태와 JSON code, 화면을 적으세요. 실제 결과 칸에는 예를 들어 403 / FORBIDDEN 처럼 기록합니다. 정상 응답은 200 / 해당 없음처럼 적습니다.

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	비로그인 목록·상세	200, 정상 공개 조회	200 / 해당 없음	PASS	T01-public
T02	Alice 정상 로그인	200, JWT 와 공개 사용자 정보, 화면 로그인 표시	200 / 해당 없음	PASS	T02-login_Header T02-login_Response
T03	비밀번호 오류 및 없는 이메일 각각 로그인	401 INVALID_CREDENTIALS, 토큰 미발급	401 / INVALID_CREDENTIALS	PASS	T03-wrong_password_Header T03-wrong_password_Response
T04	무토큰 /me·글 등록	각각 401 TOKEN_REQUIRED	401 / TOKEN_REQUIRED	PASS	T04-no_token_1 T04-no_token_2
T05	Alice 글 등록·내 정보	201, authorId 1, /me 는 Alice	201 / 해당 없음	PASS	T05-create_Header T05-create_Response T05-me_Header T05-me_Response
T06	Alice 본인 글 수정·삭제	200 및 204, 결과가 DB 에 반영	200 및 204 / 해당 없음	PASS	T06-delete_Header T06-delete_Response T06-update_Header T06-update_Response
T07	Bob 이 Alice 글 수정	403 FORBIDDEN, 원문 유지	403 / FORBIDDEN	PASS	T07-bob_buttons T07-bob_update

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T08	Bob 이 Alice 글 삭제	403 FORBIDDEN, 글 유지	403 / FORBIDDEN	PASS	T08-bob_delete T08-bob_delete_unchanged
T09	서명 변조 후 /me, 등록·수정·삭제	401 INVALID_TOKEN, 데이터 변경 없음	401 / INVALID_TOKEN	PASS	T09-tamperd_1 T09-tamperd_2
T10	실제 만료 후 /me, 등록·수정·삭제	401 TOKEN_EXPIRED, 화면 로그인 해제와 안내	401 / TOKEN_EXPIRED	PASS	T10-expiry_Header T10-expiry_Response
T11	로그아웃 후 무토큰 /me	헤더에서 토큰이 빠지고 401 TOKEN_REQUIRED	401 / TOKEN_REQUIRED	PASS	T11-logout_Header T11-logout_Response
T12	등록에 authorId, userId 각각 추가	400 BAD_INPUT, 위조 글 미생성	400 / BAD_INPUT	PASS	T12-forgery
T13	빈 값·공백·길이 초과	400 BAD_INPUT, DB 변경 없음	400 / BAD_INPUT	PASS	T13-wrong_form
T14	없는 글·잘못된 ID	없는 글 404, 잘못된 ID 400; 수정·삭제에는 정상 토큰 사용	404 / NOT_FOUND	PASS	T14-not_exist
T15	실제 서버 재시작	기존 글과 작성자·내용 유지	200 / 해당 없음	PASS	T15-server_restart T15-server_restart_log
T16	페이지 새로고침	공개 글 유지, 메모리 로그인 상태 해제	200 / 해당 없음	PASS	T16-refresh
T17	무토큰 수정·삭제 / Bob 본인 CRUD	무토큰은 401, Bob 본인 작업은 성공	401 / TOKEN_REQUIRED 및 Bob 본인은 200 / 해당 없음	PASS	T04-no_token_1 T04-no_token_2 T17-no_token_change T17-no_token_change_result

HTTP 응답 확인 기록

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 조회·로그인·수정	**200**	200/ 해당 없음	T01-public T02-login_Response T06-update_Header T06-update_Response
새 글 등록	**201 및 Location**	201 / 해당 없음	T05-create_Header T05-create_Response
본인 글 삭제	**204 및 빈 본문**	204 / 해당 없음	T06-delete_Header T06-delete_Response
입력 오류·추가 작성자 필드	**400 BAD_INPUT**	400 / BAD_INPUT	T12-forgery T13-wrong_form
잘못된 계정	**401 INVALID_CREDENTIALS**	400 / INVALID_CREDENTIALS	T03-wrong_password_Header T03-wrong_password_Response
토큰 없음·헤더 형식 오류	**401 TOKEN_REQUIRED**	401 / TOKEN_REQUIRED	T04-no_token_1 T04-no_token_2
변조·클레임 오류	**401 INVALID_TOKEN**	401 / INVALID_TOKEN	T09-tamperd_1 T09-tamperd_2
정상 서명의 만료 토큰	**401 TOKEN_EXPIRED**	401 / TOKEN_EXPIRED	T10-expiry_Header T10-expiry_Response
다른 작성자의 변경	**403 FORBIDDEN**	403 / FORBIDDEN	T07-bob_update T08-bob_delete
없는 글	**404 NOT_FOUND**	404 / NOT_FOUND	T14-not_exist

[보호 API 401 의 WWW-Authenticate: Bearer 확인 결과와 증거]

토큰 없이 me 와 등록 api 요청 시 401 과 TOKEN_REQUIRED 반환되었으며 변조된 토큰 요청 시 401 과 INVALID_TOKEN 이 반환되었다. 두 경우 모두 WWW-uthenticate: Bearer 응답 헤더를 확인하였고 보호 api 가 Bearer 토큰 인증을 요구한다는 것을 의미한다. (증거: T04-no_token_1, T09-tamperd_1)

[401 과 403 에서 화면의 차이]

401 은 유효한 인증이 없으므로 메모리의 토큰과 정보를 제거하고 재로그인을 안내한다.

403 은 인증에 성공했으나 해당 글을 변경할 권한이 없는 상태이므로 수정, 삭제 등 버튼이 표시되지 않는다.

[실패한 수정·삭제 뒤 공개 조회로 데이터 불변을 확인한 결과]

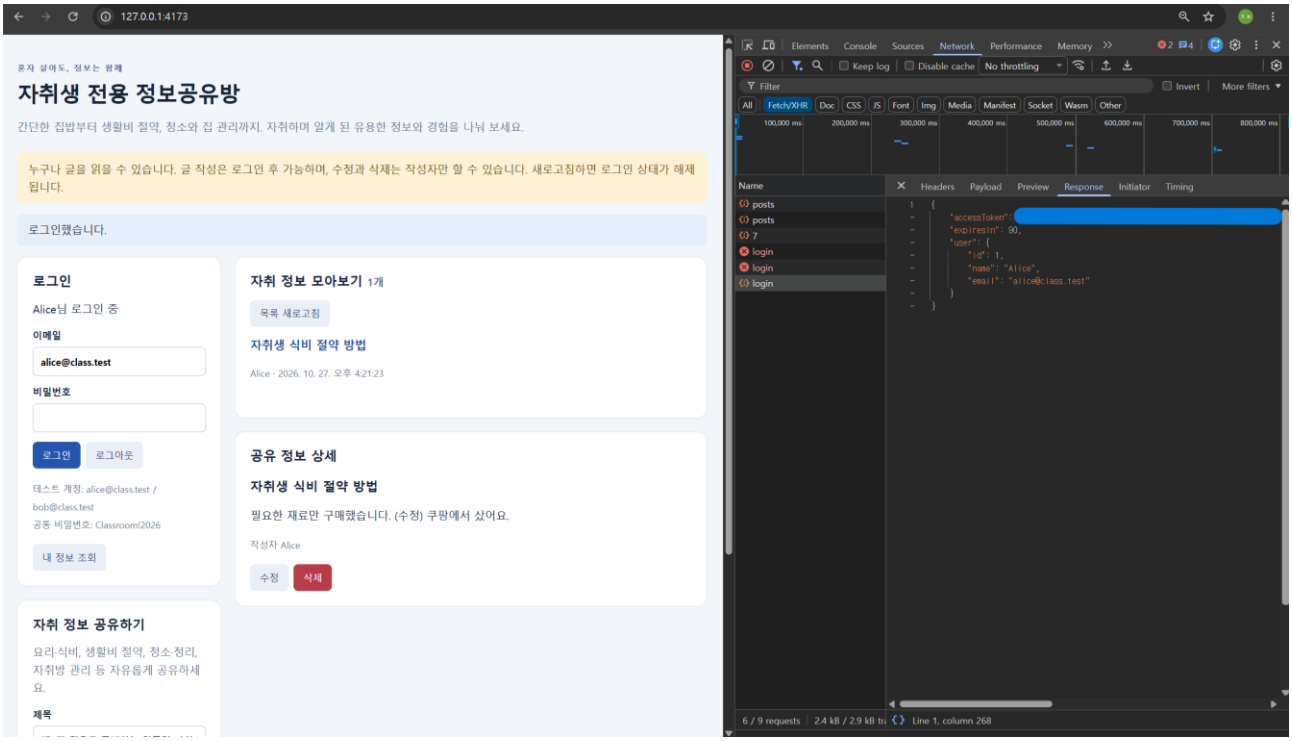
Bob 의 정상 토큰으로 Alice 게시글 수정, 삭제 요청하자 403 FORBIDDEN 으로 거부되었으며 이후 인증 없이 조회한 결과 HTTP 200, 기존 게시글이 유지되었다. (증거: T08-bob_delete_unchanged)

[토큰 없이 없는 글 수정 401 / 정상 토큰으로 없는 글 수정 404 의 실제 결과]

존재하지 않는 글에 토큰 없이 수정 요청을 보냈을 때는 401 TOKEN_REQUIRED 가 반환되었고 정상 토큰이 있는 경우에는 404 NOT_FOUND 가 반환되었다. (증거: T17-no_token_change, T14-not_exist)

테스트 증거

로그인과 토큰 전송



다른 작성자의 수정과 삭제

```
(.venv) PS C:\Users\JongHak\Desktop\JWT-Assignment> curl.exe -i -X PUT "$apiBase/posts/$alicePostId"
>>   -H "Authorization: Bearer $bobToken" `
>>   -H "Content-Type: application/json" `
>>   --data-raw '{}'
HTTP/1.1 403 Forbidden
date: Tue, 27 Oct 2026 18:38:38 GMT
server: uvicorn
content-length: 103
content-type: application/json

{"code":"FORBIDDEN","message":"본인이 작성한 글만 수정하거나 삭제할 수 있습니다."}
```

```
(.venv) PS C:\Users\JongHak\Desktop\JWT-Assignment> curl.exe -i -X DELETE "$apiBase/posts/$alicePostId"
>>   -H "Authorization: Bearer $bobToken"
HTTP/1.1 403 Forbidden
date: Tue, 27 Oct 2026 18:38:45 GMT
server: uvicorn
content-length: 103
content-type: application/json

{"code":"FORBIDDEN","message":"본인이 작성한 글만 수정하거나 삭제할 수 있습니다."}
```

토큰 변조와 만료

```
(.venv) PS C:\Users\JongHak\Desktop\JWT-Assignment> $tokenParts = $aliceToken.Split(".")
(.venv) PS C:\Users\JongHak\Desktop\JWT-Assignment> $replacement = if ($tokenParts[2][0] -eq 'A') { 'B' } else { 'A' }
(.venv) PS C:\Users\JongHak\Desktop\JWT-Assignment> $tokenParts[2] = $replacement + $tokenParts[2].Substring(1)
(.venv) PS C:\Users\JongHak\Desktop\JWT-Assignment> $badToken = $tokenParts -join "."
(.venv) PS C:\Users\JongHak\Desktop\JWT-Assignment> curl.exe -i "$apiBase/me" `
>>   -H "Authorization: Bearer $badToken"
HTTP/1.1 401 Unauthorized
date: Tue, 27 Oct 2026 18:46:39 GMT
server: uvicorn
www-authenticate: Bearer
content-length: 73
content-type: application/json

{"code":"INVALID_TOKEN","message":"유효하지 않은 토큰입니다."}
(.venv) PS C:\Users\JongHak\Desktop\JWT-Assignment> curl.exe -i -X POST "$apiBase/posts" `
>>   -H "Authorization: Bearer $badToken" `
>>   -H "Content-Type: application/json" `
>>   --data-raw '{}'
HTTP/1.1 401 Unauthorized
date: Tue, 27 Oct 2026 18:46:49 GMT
server: uvicorn
www-authenticate: Bearer
content-length: 73
content-type: application/json

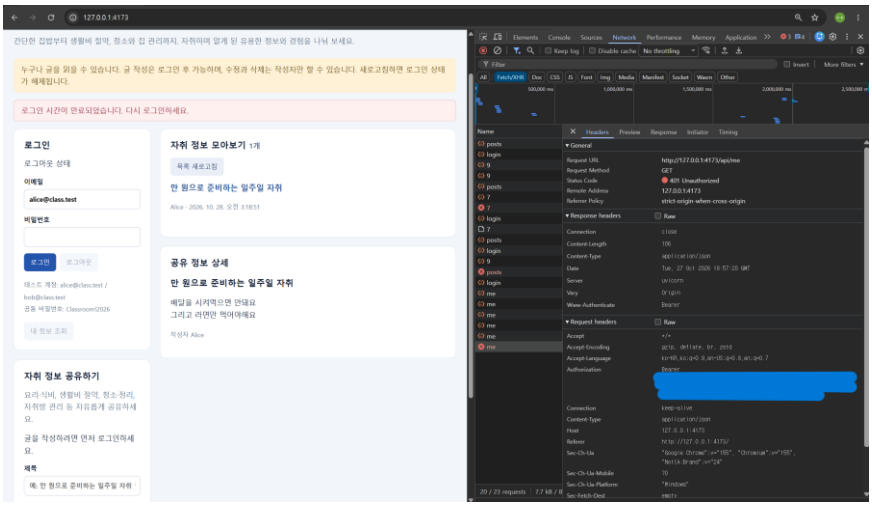
{"code":"INVALID_TOKEN","message":"유효하지 않은 토큰입니다."}
(.venv) PS C:\Users\JongHak\Desktop\JWT-Assignment> curl.exe -i -X PUT "$apiBase/posts/$alicePostId" `
>>   -H "Authorization: Bearer $badToken" `
>>   -H "Content-Type: application/json" `
>>   --data-raw '{}'
HTTP/1.1 401 Unauthorized
date: Tue, 27 Oct 2026 18:46:55 GMT
server: uvicorn
www-authenticate: Bearer
content-length: 73
content-type: application/json

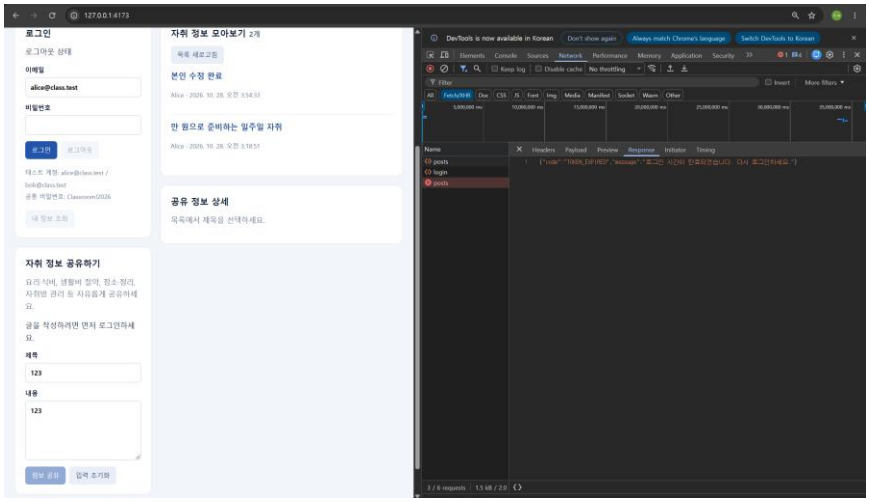
{"code":"INVALID_TOKEN","message":"유효하지 않은 토큰입니다."}
```

```
(.venv) PS C:\Users\JongHak\Desktop\JWT-Assignment> curl.exe -i -X DELETE "$apiBase/posts/$alicePostId" `
>>   -H "Authorization: Bearer $badToken"
HTTP/1.1 401 Unauthorized
date: Tue, 27 Oct 2026 18:47:05 GMT
server: uvicorn
www-authenticate: Bearer
content-length: 73
content-type: application/json

{"code":"INVALID_TOKEN","message":"유효하지 않은 토큰입니다."}
(.venv) PS C:\Users\JongHak\Desktop\JWT-Assignment> curl.exe -i "$apiBase/posts/$alicePostId"
HTTP/1.1 200 OK
date: Tue, 27 Oct 2026 18:47:41 GMT
server: uvicorn
content-length: 275
content-type: application/json

{"id":9,"title":"만 원으로 준비하는 일주일 자취","content":"배달을 시켜먹으면 안돼요\n그리고 라면만 먹어야해요","authorId":1,"authorName":"Alice","createdAt":"2026-10-27T18:18:51.012810+00:00","updatedAt":"2026-10-27T18:24:18.517390+00:00"}
(.venv) PS C:\Users\JongHak\Desktop\JWT-Assignment>
```





자동 검사와 로컬 배포 증거

.\backend-fastapi\.venv\Scripts\python.exe tools/check_api.py --mode final

```
PASS create post
PASS Bob own update
PASS missing protected PUT
PASS missing protected DELETE
PASS delete owned test post
PASS deleted post missing
PASS delete owned test post
PASS deleted post missing
All 40 checks passed (final). UI and restart tests must be performed separately.
(.venv) PS C:\Users\JongHak\Desktop\JWT-Assignment>
```

.\backend-fastapi\.venv\Scripts\python.exe tools/check_api.py --mode final –expiry

```
PASS expired PUT /posts/26
PASS expired DELETE /posts/26
PASS failed request leaves data unchanged
PASS login alice@class.test
PASS login bob@class.test
PASS delete owned test post
PASS deleted post missing
PASS delete owned test post
PASS deleted post missing
All 48 checks passed (final). UI and restart tests must be performed separately.
(.venv) PS C:\Users\JongHak\Desktop\JWT-Assignment>
```

[자동 검사 실행 결과]

위의 일반 검사는 로그인, 공개조회, 글 작성, 토큰 누락, 작성자 권한 및 입력 검증을 확인한다.

아래의 만료 검사는 TOKEN_TTL = 3 을 적용하고 서버를 재시작한 뒤 수행한다.

[빌드 성공 로그 / 4173 번 API 연결 / 재시작 유지의 화면 또는 파일 경로]

위에서부터 빌드 성공 로그, 4173 연결 로그, 서버 재시작 로그, 재시작 후 화면이다.

```
dist/assets/index-qxMUUX8I.js   228.22 kB | gzip: 71.97 kB

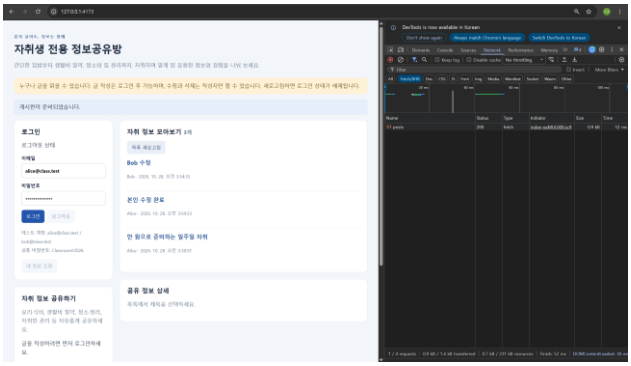
✓ built in 133ms
(.venv) PS C:\Users\JongHak\Desktop\JWT-Assignment\frontend-react> npm.cmd run preview

> jwt-board-react@1.0.0 preview
> vite preview --host 127.0.0.1

  ➜  Local:   http://127.0.0.1:4173/
  ➜  press h + enter to show help
```

```
(.venv) PS C:\Users\JongHak\Desktop\JWT-Assignment\backend-fastapi> .\.venv\Scripts\python.exe -m uvicorn app:app --host 127.0.0.1 --port 8000
INFO:     Started server process [19660]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     127.0.0.1:10603 - "GET /api/posts HTTP/1.1" 200 OK
INFO:     127.0.0.1:9820 - "GET /api/posts HTTP/1.1" 200 OK
```

```
INFO:     Finished server process [4500]
(.venv) PS C:\Users\JongHak\Desktop\JWT-Assignment\backend-fastapi> .\.venv\Scripts\python.exe -m uvicorn app:app --host 127.0.0.1 --port 8000
INFO:     Started server process [35508]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     127.0.0.1:8137 - "GET /api/posts/12 HTTP/1.1" 200 OK
```



웹서비스설계 05

AI 활용과 참고 자료

AI 사용 여부: [사용]

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
[Chat Gpt 6.1 sol]	[생성 /테스트]	backend-fastapi/auth.py Frontend-react/src/api.js, main.jsx, style.css README.md	[AI 생성 코드 반영 후 직접 실행 및 검증, 수정안 반영 후 화면 검증]

대표 사례

1.

도구 / 날짜: Codex / 2026 년 10 월 26 일

요청한 내용: "과제의 백엔드를 FastAPI 로 구현하려고 한다. auth.py 에 정의된 함수들을 기존 사용처와 API 명세에 맞게 구현해 달라"고 요청하였다.

AI 가 제안한 내용: 서버 환경 변수에서 키와 토큰 유효 시간을 읽고 HS256 으로 토큰을 발급하는 코드를 제안하였다. Bearer 토큰 추출, 서명/필수, 클레임/만료/실제 사용자 검증과 작성자 권한 검사를 구현하였다.

실제 반영 위치: backend-fastapi/auth.py: _settings(), issue_token(), current_user_id(), assert_owner()

내가 확인하고 수정한 내용: AI 가 생성한 인증코드를 확인하고 실제 API 요청을 보내 동작을 확인하였고 verify_exp=False 로 놓고 구현한 부분에 있어서 만료 검사 생략이 아닌 이후 코드에서 exp 를 별도로 현재 시각과 비교하여 검사하는 점을 확인하였다.

검증 방법과 결과: 로그인 성공 시 HTTP 200 과 토큰, 사용자 정보가 반환되는 결과를 확인하였다. 토큰 없는 요청과 변조토큰 요청이 각각 401 TOKEN_REQUIRED 와 401 INVALID_TOKEN 으로 거부되는 것을 확인하였다. 실패한 변경 요청 이후 게시글이 유지되는 것도 확인하였다.

2.

도구 / 날짜: Codex / 2026 년 10 월 26 일

요청한 내용 : "React 인증 기능을 구현하고 싶다"고 요청한 뒤, "React 화면 처리를 완성해 달라"고 추가 요청하였다.

AI 가 제안한 내용: 로그인 API 응답의 토큰과 사용자 정보를 메모리에 저장하고 요청에 Bearer 헤더를 첨부하는 코드 제안됨

실제 반영 위치: frontend-react/src/api.js: login(), authorizationHeaders(), request(), clearSession(), canEdit()

frontend-react/src/main.jsx: 로그인, 로그아웃, 내 정보, CURD 및 오류 화면

frontend-react/src/style.css: 오류 메시지와 비활성 버튼 스타일

내가 확인하고 수정한 내용: 토큰이 실제로 메모리에만 보관되는지 확인하였다. 수정 중인 글과 현재 조회 중인 글을 분리하여 다른 글을 열어도 수정 대상이 바뀌지 않도록 수정하였다. 저장, 삭제 성공 후 목록 갱신만 실패한 경우를 구분해 안내하도록 구현된 것을 확인하였다. 이후 실제 화면과 Network 응답을 확인하여 검증하였다.

검증 방법과 결과: 모의 API 오류 처리 검사를 수행할 수 있도록 AI 도구를 활용하였다. 4173 번 빌드 화면에서 로그인 표시, 헤더 전송, 글 등록과 각 요청에 대한 HTTP, JSON 을 확인하였다. Bob 으로 로그인 시 Alice 글의 수정, 삭제 버튼이 숨겨지는 것을 확인하였다. 만료된 토큰으로 정보 요청 시 401 과 만료 안내가 표시되고 로그아웃 상태로 변경되는 것을 확인하였다.

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시

이해도 확인과 회고

각 질문에 본인 코드의 동작을 근거로 답하세요.

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

[다른 사용자가 authorId 를 넣어 작성자인 것처럼 정보를 변경할 수 있다. 현재 코드에서는 추가 필드를 거부하고 검증된 토큰의 사용자 ID 로 작성자를 저장한다.]

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

[브라우저 화면을 거치지 않고 API 를 직접 요청할 수 있기 때문이다. 서버의 assert_owner()가 권한을 검사한다.]

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

[Decode 로 내용을 읽는 것 만으로는 토큰의 진위를 알 수 없다. 현재 코드에서는 서명/허용 알고리즘과 토큰 발급자, 대상, 사용자, 만료를 확인하는 검증을 거친다.]

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

[로그아웃은 메모리 세션을 지우고 만료는 서버에서 해당 토큰을 거부하게 한다. 새로고침은 프론트 메모리를 초기화 한다. 복사된 토큰은 만료 전까지 사용할 수 있다.]

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

[JWT 검증 시 토큰의 만료와 변조를 구분하는 처리를 확인하는 부분이 어려웠다. 만료 시각만 먼저 확인하면 서명 변조된 토큰도 단순 만료로 판단할 수 있기 때문이다. 이를 위해 HS256 서명과 필수 클레임, 사용자 존재 여부를 먼저 검증하고 이를 통과한 후 exp 와 현재 시각을 비교하도록 하였다. 이후 각 상황에서 서로 다른 JSON code 가 반환됨을 확인하였다.]

현재 남은 문제와 다음에 개선할 점: